L13 — Pointers and Pointer Arrays

Unit: 1 | **Chapter:** Pointers, Records & Advanced Arrays | **BT Level:** BT3 | **CO:** CO2

Learning Objectives

- Explain what a pointer is and how it relates to memory addresses
- Describe pointer arithmetic and dereferencing
- Implement pointer arrays and understand their use cases
- Distinguish between shallow copy and deep copy using pointers

---

1. What is a Pointer?

A **pointer** is a variable that stores the **memory address** of another variable rather than a direct value.

```
Memory Layout:
Address: 1000    Value: 42      ← int x = 42
Address: 2000    Value: 1000    ← int *ptr = &x   (ptr holds address of
```

In C:

```c
int x = 42;
int *ptr = &x;   // ptr stores the address of x

printf("%d", x);    // 42       — value of x
printf("%p", ptr);  // 0x1000   — address stored in ptr
printf("%d", *ptr); // 42       — dereference: value at the address
```

> **Python note:** Python doesn't expose raw pointers, but uses references internally. The `id()` function returns the memory address of an object.

---

2. Pointer Fundamentals

2.1 Declaration and Initialization (C)

```c
int  x = 10;
int  *p = &x;   // p is a pointer to int, stores address of x
int  **pp = &p; // pp is a pointer to a pointer to int

*p = 20;        // Modify x through pointer → x is now 20
**pp = 30;      // Modify x through double pointer → x is now 30
```

2.2 Pointer Arithmetic

Pointers can be incremented/decremented — they move by the size of the pointed-to type.

```
int arr[5] = {10, 20, 30, 40, 50};
int *p = arr;     // p points to arr[0]

p++;              // p now points to arr[1] (moved 4 bytes for int)
*(p + 2) = 99;   // Modifies arr[3] (arr[1] + 2 positions)

printf("%d", p[1]);   // 30 (equivalent to *(p+1))
```

Arithmetic rules:

- `ptr + k` → moves k × sizeof(type) bytes forward
- `ptr - k` → moves k × sizeof(type) bytes backward
- `ptr1 - ptr2` → number of elements between two pointers

---

3. Arrays and Pointers

In C/C++, the name of an array is a **pointer to its first element**:

```
int arr[5] = {1, 2, 3, 4, 5};
// arr == &arr[0]
// arr[i] == *(arr + i)

for (int i = 0; i < 5; i++) {
    printf("%d ", *(arr + i));  // Same as arr[i]
}
```

This is why arrays can be passed to functions as pointers with no overhead:

```
void print_array(int *arr, int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
}
```

---

4. Pointer Arrays

A **pointer array** is an array where each element is a pointer.

4.1 Array of Pointers to Integers

```
int a = 10, b = 20, c = 30;
int *pArr[3];
pArr[0] = &a;
pArr[1] = &b;
pArr[2] = &c;

for (int i = 0; i < 3; i++)
    printf("%d ", *pArr[i]);   // 10 20 30
```

4.2 Array of Strings (char pointers)

```
char *names[] = {"Alice", "Bob", "Charlie", "Diana"};

for (int i = 0; i < 4; i++)
    printf("%s\n", names[i]);
```

Memory layout:

```
names[0] → "Alice\0"
names[1] → "Bob\0"
names[2] → "Charlie\0"
names[3] → "Diana\0"
```

Each pointer takes 8 bytes (64-bit), but strings can be different lengths → **flexible storage**.

---

5. Pointer Arrays vs 2D Arrays

Feature	2D Array `char a[4][8]`	Pointer Array `char *a[4]`
Memory	Fixed: $4 \times 8 = 32$ bytes	Variable per string
Access	`a[i][j]`	`a[i][j]` (same syntax)
Row lengths	All equal	Can differ
Modifiable	In-place	Redirect pointer
Use for strings	Wastes space	Efficient

---

6. Sorting a Pointer Array

Sorting a pointer array rearranges pointers without moving data — efficient for large records:

```
# Python equivalent of pointer array sorting
students = ["Charlie", "Alice", "Diana", "Bob"]

# Sort the "pointers" (references) by the values they point to
students.sort()  # Sorts references, not the actual string data in memo
# ["Alice", "Bob", "Charlie", "Diana"]
```

In C:

```
#include <stdlib.h>
#include <string.h>

char *names[] = {"Charlie", "Alice", "Diana", "Bob"};
int n = 4;

int cmp(const void *a, const void *b) {
    return strcmp(*(char **)a, *(char **)b);
}

qsort(names, n, sizeof(char *), cmp);
// names now: ["Alice", "Bob", "Charlie", "Diana"]
```

This is much faster than moving entire records — only pointers (8 bytes) are swapped.

---

7. Dynamic Memory Allocation via Pointers

```
#include <stdlib.h>

// Allocate array at runtime
int n = 10;
int *arr = (int *)malloc(n * sizeof(int));

if (arr == NULL) {
    // Handle allocation failure
    exit(1);
}

// Use arr like a normal array
for (int i = 0; i < n; i++)
    arr[i] = i * i;

free(arr);  // Always free after use to prevent memory leaks
arr = NULL; // Good practice: avoid dangling pointer
```

---

8. NULL Pointer and Dangling Pointers

Issue	Description	Prevention
NULL pointer	Pointer not initialized, points to address 0	Always initialize pointers
Dangling pointer	Pointer to freed memory	Set to NULL after `free()`
Wild pointer	Uninitialized pointer with garbage address	Always initialize before use
Memory leak	Allocated but never freed	Call `free()` for every `malloc()`

---

9. Python Reference Model (Conceptual Pointers)

Python uses **references** (conceptually similar to pointers):

```
x = [1, 2, 3]
y = x           # y references the SAME list (shallow copy)
y.append(4)
print(x)        # [1, 2, 3, 4] — x is also modified

import copy
z = copy.deepcopy(x)  # Deep copy — independent copy
z.append(5)
print(x)        # [1, 2, 3, 4] — x unchanged
```

Summary

- A **pointer** stores a memory address, allowing indirect access to data.
- **Pointer arithmetic** moves by multiples of the element size.
- Array name = pointer to first element in C.
- **Pointer arrays** hold addresses of other data — useful for variable-length data and efficient sorting of large records.
- In Python, all variables are references (conceptual pointers).
- Common pitfalls: NULL pointers, dangling pointers, memory leaks.

References

- Cormen et al., *Introduction to Algorithms*, Ch. 10.3 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 3.1 (Springer, 2020)